SECURIFY

YOUR SECURITY PARTNER

Swiftdigital

Swiftdigital Web & API VAPT 2026

May 08, 2026

Confidentiality and Distribution Restrictions

The conclusions and recommendations in this report represent the opinions of Securify. Determinations of appropriate corrective action(s) are the responsibility of the entity receiving the report. This report and any other materials furnished by Securify in connection with this engagement are confidential and may not be duplicated, modified, or otherwise reproduced and distributed without the express prior written consent of Securify or Swiftdigital.

Table of Contents

Introduction

As part of an ongoing security program, Swiftdigital identified the need to conduct an application security assessment of its Web application & APIs.

This report presents the agreed scope, methodology, risk measurement model, summarized findings, and detailed technical observations for the selected assessment window.

The following lists the objectives of this assessment:

- Determine the overall security posture of the application
- Provide a list of key findings and recommendations for remediation
- Document the assessment scope, risk evaluation, and final outcomes
- Support remediation planning with actionable technical detail

This report includes the following parameters and results of the assessment:

Approach

Securify performed a Runtime Application Vulnerability Assessment of the client environment using a combination of manual testing, guided analysis, and targeted verification of identified attack paths.

Testing focused on authentication, authorization, business logic, session management, input handling, information disclosure, and transport-layer protections.

Where appropriate, the assessment also included abuse-case validation, access-control bypass attempts, forced browsing, parameter tampering, and endpoint enumeration.

Runtime Application Vulnerability Assessment

The runtime assessment focused on security behavior observable in the live application and API flows, including authentication, authorization, input handling, business logic, sensitive data exposure, and common attack-surface weaknesses.

The engagement emphasized practical exploitability and realistic attacker behavior rather than purely theoretical weaknesses.

- **Information Gathering** – The application was reviewed as an anonymous, authenticated, and privileged user to understand differences in access and behavior. Technologies, frameworks, APIs, and third-party integrations were also identified to support targeted testing.
- **Authentication Testing** – Authentication mechanisms were evaluated to determine the strength of login controls, password policies, and account recovery processes. Protections against brute-force attempts and session takeover scenarios were also reviewed to ensure users are securely authenticated.
- **Authorization Testing** – Tests were conducted to identify weaknesses in access control, including attempts to access other users' data or privileged functionality.
- **Session Management** – Session handling was assessed to ensure secure creation, storage, and invalidation of session tokens.
- **Input Validation Attacks** – User-controlled input fields were tested with malformed and malicious data to identify injection flaws and logic bypasses. This included attempts to exploit unexpected behavior, access unprotected functionality, or inject vulnerabilities such as XSS, SQL Injection, and Command Injection.
- **Business Logic Testing** – The application workflows were reviewed to identify opportunities to misuse or bypass intended processes. This included testing for logic errors, insufficient validation, and scenarios where typical constraints could be

circumvented.

Scope

The assessment was conducted between May 07, 2026 and May 28, 2026. The scope of the assessment was limited to the environments and targets below:

Name	URL
swiftdigital	https://swiftdigitalservices.com

Role	Username
Administrator	swiftdigitalh@gmail.com
Administrator	swiftdigitald@gmail.com

Tool Name	Description
Burp Professional Pro	An advanced proxy for testing web security.
OpenSSL	An open-source package to assess security in transit.
Nmap	A tool used to discover hosts and services on a network.
SQLMap	Python-based CLI tool that identifies SQL injection issues.
Acunetix Pro	An automated scanner to perform authenticated scans on the web app / APIs.
cURL Utility	Command line utility to perform HTTP/s requests.

The following components and tests were out of scope for this review:

Any applications and infrastructure external to the Swiftdigital's Web Application & API.

In cases where the Swiftdigital's Web Application & API had inbound and/or outbound interfaces with:

Another application, the interfaces, and communications were considered in scope.

All other external elements were excluded.

Supporting policies, procedures, and processes.

Social Engineering.

Software Development Life Cycle.

Assessment Limitation

This assessment was performed within the agreed timebox and only against systems explicitly included in scope. Absence of identified vulnerabilities should not be interpreted as a guarantee that no issues remain.

Findings and Recommendation

Risk Classification

Risk is determined by evaluating the combined effect of impact and likelihood for each issue identified during testing.

The matrix below is used as the standard model for determining overall severity.

Measurement of Impact

Factor	Description	Effect on Impact
Classification of Data Loss	If an attacker exploited this vulnerability, what type of data would they access or steal? Non-sensitive, non-customer-specific data would lower the impact.	Less critical data reduces the overall impact.
Data Integrity	Could the attacker modify or delete data they shouldn't? If yes, what kind of data could they corrupt?	Non-critical or limited modification rights reduce the impact of exploitation.
Blast Radius	How many users or systems would be affected by this vulnerability? For instance, an obscure service affects fewer users, while a public system affects many.	A small number of victims lowers the severity.

Impact reflects the potential business, technical, operational, and data-related consequences if a vulnerability is successfully exploited.

- **Critical Impact:** Severe compromise of confidentiality, integrity, or availability with material business or operational consequence.
- **High Impact:** Significant unauthorized access, data exposure, or business-process abuse affecting sensitive functions or users.
- **Medium Impact:** Meaningful security weakness with constrained scope, partial exposure, or limited operational effect.
- **Low Impact:** Minor weakness with limited practical consequence or low-value exposure.

Measurement of Likelihood

Factor	Description	Effect on Likelihood
Required Skill Level of Attacker	How technically skilled does the attacker need to be to exploit the vulnerability? The higher the required skills, the lower the likelihood.	Complex skill requirements lower the chances of exploitation.
Number of Potential Attackers	How many attackers could potentially exploit this vulnerability? If access to the system is limited, fewer attackers can exploit it.	Fewer attackers reduce the probability of exploitation.
Cost of Exploit	How expensive are the tools or resources required for exploitation? High costs reduce accessibility for attackers.	Expensive tools lower the likelihood of exploitation.
Ease of Exploit	How simple is it for the attacker to exploit the vulnerability once discovered? Does it rely on external events or tools?	Complex or multi-step scenarios reduce the chances of successful exploitation.

Likelihood reflects how feasible exploitation is in practice, considering attacker capability, required access, and environmental constraints.

- **High Likelihood:** Exploitation is straightforward and requires limited skill, access, or environmental dependency.
- **Medium Likelihood:** Exploitation is feasible but requires some knowledge, sequencing, or favorable conditions.
- **Low Likelihood:** Exploitation requires significant effort, elevated preconditions, or specialized capability.

Overall Risk

Overall risk is derived from the intersection of impact and likelihood and is used to prioritize remediation efforts. Findings with the highest combined rating should be addressed first, especially where exploitability and business consequence are both significant.

- Low Impact + Low Likelihood = Low Risk
- High Impact + High Likelihood = Critical Risk

Medium	High	Critical
Low	Medium	High
Low	Low	Medium

Zero-risk Issues

Items categorized as zero-risk or informational are included where useful for context but should not be treated as exploitable vulnerabilities. These entries may still be valuable for hardening, visibility, or long-term security maturity planning.

Vulnerabilities

Summary

Below is a summary of the vulnerabilities discovered during the assessment, ranked in order of risk as determined.

Vulnerability	Risk
Broken Authentication: Missing Session Validation for Zendesk JWT Generation	**High**
Broken Access Control Allows Unauthorized User Data Access	**Medium**

Detailed Vulnerabilities

Broken Authentication: Missing Session Validation for Zendesk JWT Generation

Risk: High

Description:

The application's Zendesk messaging JWT generation endpoint, located at `https://swiftdigitalservices.com/ajax/zendesk_messaging_jwt.php`, is vulnerable to broken authentication. This endpoint accepts a POST request containing a base64-encoded user ID to generate a Zendesk JWT authentication token. However, the server fails to validate the user's session cookie, allowing an unauthenticated attacker to send requests without any session information and generate valid Zendesk JWT tokens for arbitrary user IDs. These tokens can then be used to access user-specific information and functionality within the Zendesk chat system.

Affected URL:

• https://swiftdigitalservices.com/ajax/zendesk_messaging_jwt.php

Impact and Likelihood:

- Impact:

The impact of this vulnerability is high, as an attacker can generate Zendesk JWT tokens for any user. This allows for unauthorized access to user-specific data, including Personally Identifiable Information (PII), and the ability to perform actions such as initiating or participating in chats on behalf of other users. Such unauthorized access can lead to severe privacy breaches, impersonation, reputational damage, and potential regulatory non-compliance for the affected organization.

- Likelihood:

The likelihood of exploitation is high because the vulnerability can be triggered by sending a simple POST request to the affected endpoint without requiring any prior authentication or valid session. An attacker can easily enumerate or guess user IDs, encode them, and generate valid Zendesk JWT tokens, making the attack straightforward and highly reproducible.

Steps to Reproduce:

Step 1: Identify a valid user ID within the application (e.g., by observing legitimate traffic or enumeration).

Step 2: Base64 encode the target user ID.

Step 3: Send a POST request to `https://swiftdigitalservices.com/ajax/zendesk_messaging_jwt.php` with the encoded user ID in the request body, ensuring no session cookies are included.

Step 4: Observe that the server responds with a valid Zendesk JWT authentication token for the specified user ID.

Step 5: Utilize the generated JWT token to access Zendesk functionalities as the impersonated user.

Recommendations:

- **Authentication and Authorization:** Implement robust authentication and authorization checks on the `zendesk_messaging_jwt.php` endpoint. Ensure that only authenticated and authorized users can request JWT tokens, and only for their own user ID.

- **Session Validation:** Validate the user's session cookie for all requests to the JWT generation endpoint. The server must verify that the request originates from a legitimate, authenticated user session.

- **Access Control:** Implement strict access control mechanisms to prevent unauthorized users from generating JWT tokens for other users. This may involve mapping the authenticated user's ID to the ID used in the JWT generation request.

- **Secure JWT Generation:** Review the JWT generation process to ensure it adheres to best practices, including using strong cryptographic algorithms, appropriate key management, and short expiration times for tokens.

References:

- https://owasp.org/www-project-top-ten/2021/A07_2021-Identification_and_Authentication_Failures

Broken Access Control Allows Unauthorized User Data Access

Risk: Medium

Description:

The application exhibits a broken access control vulnerability where a lower-privileged user can bypass intended authorization mechanisms. By directly accessing a specific PHP endpoint and manipulating URL parameters, an authenticated user can view details of other users, including sensitive information such as usernames and passwords, despite the user interface restricting such access. This allows for unauthorized data exposure and potential account compromise.

Affected URL:

- https://www.dev.blackrockgalleries.com/auction-admin/add_admin.php?id=137&brg=kkbiv65212

Impact and Likelihood:

- Impact:

The potential impact of this vulnerability is significant, as it can lead to unauthorized access to other user accounts within the system. Exposure of usernames and passwords can result in account takeover, privilege escalation, and further compromise of the application or associated systems. This could lead to data breaches, reputational damage, and potential regulatory non-compliance.

- Likelihood:

The vulnerability is highly likely to be exploited as it requires only an authenticated lower-privileged user to directly access a specific endpoint with a modified parameter. No complex tools or advanced techniques are necessary, making it easily discoverable and exploitable by an attacker with basic understanding of web application functionality.

Steps to Reproduce:

Step 1: Log in to the seller admin portal with a lower-privileged user account (e.g., 'manager' role).

Step 2: Navigate directly to the URL: `https://www.dev.blackrockgalleries.com/auction-admin/add_admin.php?id=137&brg=kkbiv65212` (or any other valid user ID in the 'id' parameter).

Step 3: Observe that the page displays the details of the user corresponding to the 'id' parameter, including their username and password, despite the logged-in user's lower privileges.

Recommendations:

- **Implement Server-Side Access Control:** Ensure that all requests to sensitive resources are authorized on the server-side. Before displaying or processing user data, verify that the authenticated user has the necessary permissions to access that specific resource or user ID.

- **Enforce Least Privilege:** Design access control policies based on the principle of least privilege, ensuring users only have access to the data and functionality strictly required for their role.

- **Validate Input Parameters:** Implement robust input validation for all parameters, especially those used to identify resources (e.g., 'id'). While validation helps, it should always be complemented by server-side authorization checks.

- **Avoid Direct Object References:** Do not expose internal object references (like database IDs) directly in URLs or forms. Consider using opaque, non-sequential identifiers or mapping user-specific data without exposing underlying IDs.

References:

- https://owasp.org/www-project-top-ten/2017/A5_2017-Broken_Access_Control
- https://cwe.mitre.org/data/definitions/284.html

Appendix A

Penetration Test Scope	
V1	Authentication Verification Requirements
V1.1	Password Security Requirements
1.1.1	Verify that user-set passwords are at least 12 characters in length
1.1.2	Verify that passwords 64 characters or longer are permitted
1.1.3	Verify that passwords can contain spaces and that truncation is not performed. Consecutive multiple spaces MAY optionally be coalesced
1.1.4	Verify that Unicode characters are permitted in passwords. A single Unicode code point is considered a character, so 12 emoji or 64 kanji characters should be valid and permitted
1.1.5	Verify users can change their password
1.1.7	Verify that passwords submitted during account registration, login, and password change are checked against a set of breached passwords either locally (such as the top 1,000 or 10,000 most common passwords that match the system's password policy) or using an external API. If using an API a zero-knowledge proof or other mechanism should be used to ensure that the plain text password is not sent or used in verifying the breach status of the password. If the password is breached, the application must require the user to set a new non non-breached password
1.1.8	Verify that a password strength meter is provided to help users set a stronger password
1.1.9	Verify that there are no password composition rules limiting the type of characters permitted. There should be no requirement for upper or lower case or numbers or special characters
1.1.10	Verify that "paste" functionality, browser password helpers, and external password managers are permitted
1.1.11	Verify that the user can choose to either temporarily view the entire masked password, or temporarily view the last typed character of the password on platforms that do not have this as native functionality
V1.2	General Authenticator Requirements

1.2.1	Verify that anti-automation controls are effective at mitigating breached credential testing, brute force, and account lockout attacks. Such controls include blocking the most common breached passwords, soft lockouts, rate limiting, CAPTCHA, ever-increasing delays between attempts, IP address restrictions, or risk-based restrictions such as location, first login on a device, recent attempts to unlock the account, or similar. Verify that no more than 100 failed attempts per hour is possible on a single account
1.2.2	Verify that the use of weak authenticators (such as SMS and email) is limited to secondary verification and transaction approval and not as a replacement for more secure authentication methods. Verify that stronger methods are offered before weak methods, that users are aware of the risks, or that proper measures are in place to limit the risks of account compromise
1.2.3	Verify that secure notifications are sent to users after updates to authentication details, such as credential resets, email or address changes, and logging in from unknown or risky locations. The use of push notifications - rather than SMS or email - is preferred, but in the absence of push notifications, SMS or email is acceptable as long as no sensitive information is disclosed in the notification
1.2.4	Verify impersonation resistance against phishing, such as the use of multi-factor authentication, cryptographic devices with intent (such as connected keys with a push to authenticate), or at higher ALL levels, client-side certificates
1.2.5	Verify that where a credential service provider (CSP) and the application verifying authentication are separated, mutually authenticated TLS is in place between the two endpoints
V1.3	Credential Recovery Requirements
1.3.1	Verify that a system-generated initial activation or recovery secret is not sent in clear text to the user
1.3.2	Verify password hints or knowledge-based authentication (so-called "secret questions") are not present
1.3.3	Verify password credential recovery does not reveal the current password in any way
1.3.4	Verify shared or default accounts are not present (e.g. "root", "admin", or "sa").
1.3.5	Verify that if an authentication factor is changed or replaced, the user is notified of this event
1.3.6	Verify forgotten passwords, and other recovery paths using a secure recovery mechanism, such as TOTP or another soft token, mobile push, or another offline recovery mechanism

1.3.7	Verify that if OTP or multi-factor authentication factors are lost, that evidence of identity proofing is performed at the same level as during enrolment
V1.4	Single or Multi-Factor One Time Verifier Requirements
1.4.1	Verify that time-based OTPs have a defined lifetime before expiring
1.4.2	Verify that symmetric keys used to verify submitted OTPs are highly protected, such as by using a hardware security module or secure operating system-based key storage
1.4.3	Verify that time-based OTP can be used only once within the validity period
1.4.4	Verify that if a time-based multi-factor OTP token is re-used during the validity period, it is logged and rejected with secure notifications being sent to the holder of the device
V2	Session Management Verification Requirements
V2.1	Fundamental Session Management Requirements
2.1.1	Verify the application never reveals session tokens in URL parameters or error messages
2.1.1	Verify the application generates a new session token on user authentication
2.1.2	Verify that session tokens possess at least 64 bits of entropy
2.1.3	Verify the application only stores session tokens in the browser using secure methods such as appropriately secured cookies or HTML 5 session storage
V2.2	Session Logout and Timeout Requirements
2.2.1	Verify that logout and expiration invalidate the session token, such that the back button or a downstream relying party does not resume an authenticated session, including across relying parties
2.2.2	If authenticators permit users to remain logged in, verify that re-authentication occurs periodically both when actively used or after an idle period. (C6) 30 days 12 hours or 30 minutes of inactivity, 2FA optional 12 hours or 15 minutes of inactivity, with 2FA
2.2.3	Verify that the application terminates all other active sessions after a successful password change and that this is effective across the application, federated login (if present), and any relying parties
2.2.4	Verify that users are able to view and log out of any or all currently active sessions and devices
V2.3	Cookie-based Session Management
2.3.1	Verify that cookie-based session tokens have the 'Secure' attribute set

2.3.2	Verify that cookie-based session tokens have the 'HttpOnly' attribute set
2.3.3	Verify that cookie-based session tokens utilize the 'SameSite' attribute to limit exposure to cross-site request forgery attacks
2.3.4	Verify that cookie-based session tokens use "__Host-" prefix (see references) to provide session cookie confidentiality
2.3.5	Verify that if the application is published under a domain name with other applications that set or use session cookies that might override or disclose the session cookies, set the path attribute in cookie-based session tokens using the most precise path possible
V2.4	Token-based Session Management
2.4.2	Verify the application uses session tokens rather than static API secrets and keys, except with legacy implementations
2.4.3	Verify that stateless session tokens use digital signatures, encryption, and other countermeasures to protect against tampering, enveloping, replay, null cipher, and key substitution attacks
V3	Access Control Verification Requirements
V3.1	Operation Level Access Control
3.1.2	Verify that the application or framework enforces a strong anti-CSRF mechanism to protect authenticated functionality, and effective anti-automation or anti-CSRF protects unauthenticated functionality
V4	Validation, Sanitization, and Encoding Verification Requirements Control Objective
V4.1	Input Validation Requirements
4.1.1	Verify that the application has defenses against HTTP parameter pollution attacks, particularly if the application framework makes no distinction about the source of request parameters (GET, POST, cookies, headers, or environment variables)
4.1.2	Verify that frameworks protect against mass parameter assignment attacks, or that the application has countermeasures to protect against unsafe parameter assignment, such as marking fields private or similar
4.1.3	Verify that all input (HTML form fields, REST requests, URL parameters, HTTP headers, cookies, batch files, RSS feeds, etc) is validated using positive validation (whitelisting)
4.1.4	Verify that structured data is strongly typed and validated against a defined schema including allowed characters, length and pattern (e.g. credit card numbers or telephone, or validating that two related fields are reasonable, such as checking that suburb and zip/postcode match)

4.1.5	Verify that URL redirects and forwards only allow whitelisted destinations, or show a warning when redirecting to potentially untrusted content
4.1.6	Verify that all untrusted HTML input from WYSIWYG editors or similar is properly sanitized with an HTML sanitizer library or framework feature
4.1.7	Verify that unstructured data is sanitized to enforce safety measures such as allowed characters and length
4.1.8	Verify that the application sanitizes user input before passing to mail systems to protect against SMTP or IMAP injection
4.1.9	Verify that the application avoids the use of eval() or other dynamic code execution features. Where there is no alternative, any user input being included must be sanitized or sandboxed before being executed
4.1.10	Verify that the application protects against template injection attacks by ensuring that any user input being included is sanitized or sandboxed
4.1.11	Verify that the application protects against SSRF attacks, by validating or sanitizing untrusted data or HTTP file metadata, such as filenames and URL input fields, using whitelisting of protocols, domains, paths, and ports
4.1.12	Verify that the application sanitizes, disables, or sandboxes user-supplied SVG scriptable content, especially as they relate to XSS resulting from inline scripts, and foreign Objects
4.1.13	Verify that the application sanitizes, disables, or sandboxes user-supplied scriptable or expression template language content, such as Markdown, CSS or XSL stylesheets, BBCode, or similar
V4.2	Output encoding and Injection Prevention Requirements
4.2.1	Verify that output encoding is relevant for the interpreter and context required. For example, use encoders specifically for HTML values, HTML attributes, JavaScript, URL Parameters, HTTP headers, SMTP, and others as the context requires, especially from untrusted inputs (e.g. names with Unicode or apostrophes, such as ねこ or O'Hara).
4.2.2	Verify that output encoding preserves the user's chosen character set and locale, such that any Unicode character point is valid and safely handled
4.2.3	Verify that context-aware, preferably automated - or at worst, manual - output escaping protects against reflected, stored, and DOM-based XSS
4.2.4	Verify that data selection or database queries (e.g. SQL, HQL, ORM, NoSQL) use parameterized queries, ORMs, entity frameworks, or are otherwise protected from database injection attacks

4.2.5	Verify that where parameterized or safer mechanisms are not present, context-specific output encoding is used to protect against injection attacks, such as the use of SQL escaping to protect against SQL injection
4.2.6	Verify that the application projects against JavaScript or JSON injection attacks, including for eval attacks, remote JavaScript includes, CSP bypasses, DOM XSS, and JavaScript expression evaluation.
4.2.7	Verify that the application protects against LDAP Injection vulnerabilities, or that specific security controls to prevent LDAP Injection have been implemented
4.2.8	Verify that the application protects against OS command injection and that operating system calls use parameterized OS queries or use contextual command line output encoding
4.2.9	Verify that the application protects against Local File Inclusion (LFI) or Remote File Inclusion (RFI) attacks.
4.2.10	Verify that the application protects against XPath injection or XML injection attacks
V4.3	Deserialization Prevention Requirements
4.3.1	Verify that serialized objects use integrity checks or are encrypted to prevent hostile object creation or data tampering
4.3.2	Verify that the application correctly restricts XML parsers to only use the most restrictive configuration possible and to ensure that unsafe features such as resolving external entities are disabled to prevent testE
4.3.3	Verify that deserialization of untrusted data is avoided or is protected in both custom code and third-party libraries (such as JSON, XML and YAML parsers)
4.3.4	Verify that when parsing JSON in browsers or JavaScript-based backends, JSON.parse is used to parse the JSON document. Do not use eval() to parse JSON
V5	Error Handling and Logging Verification Requirements
V5.1	Error Handling
5.1.1	Verify that a generic message is shown when an unexpected or security sensitive error occurs, potentially with a unique ID which support personnel can use to investigate
V6	Data Protection Verification Requirements
V6.1	Sensitive Private Data
6.1.1	Verify that sensitive data is sent to the server in the HTTP message body or headers, and that query string parameters from any HTTP verb do not contain sensitive data
V7	Communications Verification Requirements

V7.1	Communications Security Requirements
7.1.1	Verify that secured TLS is used for all client connectivity, and does not fall back to insecure or unencrypted protocols.
7.1.2	Verify using online or up-to-date TLS testing tools that only strong algorithms, ciphers, and protocols are enabled, with the strongest algorithms and ciphers set as preferred
7.1.3	Verify that old versions of SSL and TLS protocols, algorithms, ciphers, and configurations are disabled, such as SSLv2, SSLv3, or TLS 1.0 and TLS 1.1. The latest version of TLS should be the preferred cipher suite
V8	Business Logic Verification Requirements
8.1.1	Verify the application will only process business logic flows for the same user in sequential step order and without skipping steps
8.1.2	Verify the application will only process business logic flows with all steps being processed in realistic human time, i.e. transactions are not submitted too quickly
8.1.3	Verify the application has appropriate limits for specific business actions or transactions which are correctly enforced on a per-user basis
8.1.4	Verify the application has sufficient anti-automation controls to detect and protect against data exfiltration, excessive business logic requests, excessive file uploads, or denial of service attacks
8.1.5	Verify the application has business logic limits or validation to protect against likely business risks or threats, identified using threat modeling or similar methodologies
8.1.6	Tested for IDOR's vulnerabilities which would allow other users to retrieve any kind of sensitive information like api key, tokens etc.
8.1.7	Verified Privilege Escalation vulnerabilities which could allow low privilege users to perform arbitrary operations leading potential account takeovers.
8.1.8	Verified Privilege Escalation vulnerabilities which could allow users to perform CRUD Operations on behalf of victim users.
8.1.9	Checked for Information Disclosure vulnerabilities via IDORs which could result in users PII Disclosure (PII).
8.1.10	Checked for Broken Access Control issues via browsed forcing.
8.1.11	Checked for Mass Assignment vulnerability which could allow attackers to gain extra privileges.
V9	File and Resources Verification Requirements

V9.1	File Upload Requirements
9.1.1	Verify that the application will not accept large files that could fill up storage or cause a denial of service attack
V9.2	File Integrity Requirements
9.2.1	Verify that files obtained from untrusted sources are validated to be of expected type based on the file's content
V10	API and Web Service Verification Requirements
V10.1	Generic Web Service Security Verification Requirements
10.1.1	Verify that all application components use the same encodings and parsers to avoid parsing attacks that exploit different URI or file parsing behavior that could be used in SSRF and RFI attacks
10.1.2	Verify that access to administration and management functions is limited to authorized administrators
10.1.3	Verify API URLs do not expose sensitive information, such as the API key, session tokens, etc
10.1.4	Verify that authorization decisions are made at both the URI, enforced by programmatic or declarative security at the controller or router, and at the resource level, enforced by model-based permissions
10.1.5	Verify that requests containing unexpected or missing content types are rejected with appropriate headers (HTTP response status 406 Unacceptable or 415 Unsupported Media Type)
V10.2	RESTful Web Service Verification Requirements
V11	Configuration Verification Requirements
V11.1	Dependency
11.1.1	Verify that all components are up to date
11.1.2	Verify that all unneeded features, documentation, samples, configurations are removed, such as sample applications, platform documentation, and default or example users
11.1.3	Verify that if application assets, such as JavaScript libraries, CSS stylesheets or web fonts, are hosted externally on a content delivery network (CDN) or external provider, Subresource Integrity (SRI) is used to validate the integrity of the asset
V11.2	Unintended Security Disclosure Requirements

11.2.1	Verify that web or application server and framework error messages are configured to deliver user-actionable, customized responses to eliminate any unintended security disclosures
11.2.2	Verify that web or application server and application framework debug modes are disabled in production to eliminate debug features, developer consoles, and unintended security disclosures
11.2.3	Verify that the HTTP headers or any part of the HTTP response do not expose detailed version information of system components
V11.3	HTTP Security Headers Requirements
11.3.1	Verify that every HTTP response contains a content type header specifying a safe character set (e.g., UTF-8, ISO 8859-1).
11.3.2	Verify that all API responses contain Content-Disposition: attachment; filename="api.json" (or other appropriate filename for the content type).
11.3.3	Verify that a content security policy (CSPv2) is in place that helps mitigate impact of XSS attacks like HTML, DOM, JSON, and JavaScript injection vulnerabilities
11.3.4	Verify that all responses contain X-Content-Type-Options: nosniff
11.3.5	Verify that HTTP Strict Transport Security headers are included on all responses and for all subdomains, such as Strict-Transport-Security: max-age=15724800; include subdomains
11.3.6	Verify that a suitable "Referrer-Policy" header is included, such as "no-referrer" or "same-origin
11.3.7	Verify that a suitable X-Frame-Options or Content-Security-Policy: frame ancestors header is in use for sites where content should not be embedded in a third-party site
V11.4	Validate HTTP Request Header Requirements
11.4.1	Verify that the application server only accepts the HTTP methods in use by the application or API, including pre-flight OPTIONS
11.4.2	Verify that the supplied Origin header is not used for authentication or access control decisions, as the Origin header can easily be changed by an attacker
11.4.3	Verify that the cross-domain resource sharing (CORS) Access-Control-AllowOrigin header uses a strict white list of trusted domains to match against and does not support the "null" origin
11.4.4	Verify that HTTP headers added by a trusted proxy or SSO devices, such as a bearer token, are authenticated by the application